

Synchronization in linux kernel programming

Concurrency

What is Concurrency?

Concurrency means **multiple tasks making progress during the same period of time.**

The tasks may:

- Run one after another very quickly on a single CPU (context switching).
- Run simultaneously on multiple CPU cores.

How to find how many cores you have?

1. Nproc
2. Grep -c ^processor /proc /cpuinfo

How to find processor number in which process is running ?

Ps -eaF

```
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$ cat /proc/cpuinfo | grep
processor
processor      : 0
processor      : 1
processor      : 2
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$
```

```
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$ nproc
3
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$ grep -c ^processor /proc
cpuinfo
3
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$ ps -eaF
UID          PID    PPID  C   SZ   RSS  PSR  STIME  TTY          TIME CMD
root          1         0  0 41666 11984    1  Jun19 ?           00:00:03 /sbin
root          2         0  0      0      0    2  Jun19 ?           00:00:00 [kthr
root          3         2  0      0      0    0  Jun19 ?           00:00:00 [pool
root          4         2  0      0      0    0  Jun19 ?           00:00:00 [kwo
root          5         2  0      0      0    0  Jun19 ?           00:00:00 [kwo
root          6         2  0      0      0    0  Jun19 ?           00:00:00 [kwo
root          7         2  0      0      0    0  Jun19 ?           00:00:00 [kwo
```

What is a Multiprocessing System?

A multiprocessing system contains **multiple CPUs (processors)** working together in a computer system.

Multiprocessing is a system containing multiple CPUs that execute tasks concurrently. In the simplest design, each CPU has its own operating system and private memory. This improves performance but causes memory wastage, communication overhead, and synchronization

issues. Modern systems therefore use a shared operating system and shared memory model called **Symmetric Multiprocessing (SMP)**.

Symmetric Multiprocessing (SMP)

Symmetric Multiprocessing (SMP) is a multiprocessing architecture where **multiple CPUs (or cores) share the same memory and run a single operating system**.

All processors are considered **equal (symmetric)** and can perform any task.

Memory → cpu1,cpu2,cpu3,cpu4 → one OS

All CPUs:

- Share the same RAM
- Share the same OS kernel
- Share I/O devices
- Can execute any process/thread

Advantages:

1. Better Performance
2. Shared Memory
3. Efficient resource utilization
4. Single operating system
5. Easier Communication

Disadvantages

1. Synchronization problems
2. Cache Coherency Overhead
3. Memory Contention
4. Scalability Limits

Feature	SMP	AMP (Asymmetric Multiprocessing)
OS	One shared OS	Separate OS per CPU
Memory	Shared	Usually separate
CPU Role	Equal	Master/Slave
Scheduling	Any CPU can run any task	Fixed responsibilities
Communication	Easy	More complex
Flexibility	High	Lower

What is Preemption?

Preemption means the Linux kernel can **stop a currently running task and switch the CPU to another higher-priority task** before the current task finishes.

The urgent task preempts the less important one.

forcefully taking away of the processor from one process and allocating it to another process.

2. What is Context Switching?

A **context switch** is the operation of saving the state of one task and restoring the state of another task.

The kernel saves:

Program Counter (PC)

CPU Registers

Stack Pointer (SP)

Process State

What is preemption?

- Preemption means forcefully taking away the processor from one process and allocating it to another process.
- Switching from one running task/process to another process is known as a context switch.

How is it implemented on the Linux Kernel?

- The scheduler is called after each timer interrupt (occurring many times per second).
- The scheduler determines which process to run next based on factors like priority and time already run.

Difference between preemption and context switch?

- Preemption: This occurs when the firing of a timer interrupt preempts the currently running process to run the Interrupt Service Routine (ISR) of that timer interrupt.
- Context Switch: This is the specific action where the kernel alters the state of the processor—including registers, mode, and stack—between one process or thread's context and another.
- The `context_switch()` function is called within the kernel to perform this operation.

Preemption in Linux

What is Preemption?

Preemption means:

The operating system can stop a currently running task and allow another task to run.

The running task does **not** voluntarily give up the CPU.

User Space Preemption

User processes have always been preemptible.

Example:

```
while(1)
{
    // infinite loop
}
```

Even though this process never exits the loop:

Timer Interrupt



Scheduler Runs



Another Process Gets CPU

So the system remains responsive.

Why?

Because Linux periodically generates timer interrupts and the scheduler can choose another process.

Kernel Space Preemption (Before Linux 2.6)

Older Linux kernels were **non-preemptive** inside the kernel.

When a process entered kernel mode:

```
read();
write();
open();
```

the kernel would continue executing until:

1. The system call finished, OR
2. The code explicitly called:

```
schedule();
```

Only then could another task run.

Example

Process A



System Call



Kernel Mode



Cannot be Preempted

Even if a higher-priority task became ready:

Process B (High Priority)

it had to wait.

Problem with Non-Preemptive Kernel

1. High Latency

Suppose:

Task A enters kernel

and spends:

50 ms

100 ms

200 ms

inside kernel code.

Other tasks must wait.

This increases response time (latency).

2. Poor Real-Time Performance

Real-time tasks need immediate CPU access.

Example:

Audio Processing

Video Streaming

Industrial Control

If the kernel is busy:

High Priority Task

↓

Cannot Run Immediately

Deadlines may be missed.

3. Infinite Loop in Kernel

User-space infinite loop:

```
while(1);
```

can be preempted.

Kernel-space infinite loop:

```
while(1);
```

could freeze the entire system.

Kernel Loop



No schedule()



CPU Never Released



System Hang

Linux 2.6 and Later

Linux introduced **kernel preemption**.

Now even while executing kernel code:

Kernel Task Running



Higher Priority Task Ready



Kernel Can Be Preempted



Scheduler Switches Tasks

This greatly improves:

- Responsiveness
- Latency
- Scalability
- Real-time behavior

Before Linux 2.6, user-space processes were preemptible, but kernel code was non-preemptive. Once a thread entered the kernel, it could continue running until the system call completed or it explicitly called `schedule()`. This caused high latency and poor scalability. Modern Linux kernels support kernel preemption, allowing higher-priority tasks to preempt code running in kernel mode, improving responsiveness and real-time performance. The actual switching of CPU state between tasks is performed by `context_switch()` after the scheduler decides that a preemption should occur.

CONFIG PREEMPT By using this option we can enable or disable the kernel preemption. Once enable this option, then kernel code can be preempted everywhere.

Kernel preemption can occur:

- When returning to kernel-space from an interrupt handler
- When kernel code becomes preemptible again
- If a task in the kernel explicitly calls `schedule()`
- If a task in the kernel blocks (which results in a call to `schedule()`)

Reentrancy

A function or code is called **reentrant** if it can be safely executed by multiple threads/processes at the same time without causing data corruption or unexpected behavior.

What is kernel control path?

A kernel control path denotes the sequence of instructions executed by the kernel to handle a system call, an exception, or an interrupt.

Reentrant Kernel

A **reentrant kernel** is a kernel that allows multiple processes or threads to execute kernel code simultaneously.

Why Is Reentrancy Needed?

Imagine a 4-core machine:

CPU1 → `read()`

CPU2 → `write()`

CPU3 → `open()`

CPU4 → `ioctl()`

All CPUs may enter the kernel at the same time.

Without reentrancy:

Only one CPU could execute kernel code
which wastes performance.

Problem Created by Reentrancy

Since multiple CPUs can execute kernel code simultaneously:

This leads to:

- Race conditions
- Data corruption
- Inconsistent kernel data structures

How Linux Achieves Reentrancy

Linux protects shared kernel data using synchronization mechanisms.

Spin locks, mutex, semaphore, Atomic operations

Synchronizartion and critical regions

implementing Reentrant kernel requires the use of synchronization

Why Synchronization is Needed?

Modern Linux kernels are **reentrant**, meaning multiple CPUs/processes can execute kernel code at the same time.

If two kernel control paths access the same data simultaneously, the data may become corrupted.

Synchronization is used to protect shared data and maintain consistency.

Race Condition

A **race condition** occurs when the result of a computation depends on the order in which processes or threads execute.

Example

Suppose:

$V = 1$;

Two kernel control paths execute:

Control Path A	Control Path B
----------------	----------------

-----	-----
Read $V = 1$	Read $V = 1$
$V = V + 1$	$V = V + 1$
Write $V = 2$	Write $V = 2$

Final value:

$V = 2$

Expected value:

$V = 3$

This incorrect result is called a **race condition**.

Critical Region (Critical Section)

A **critical region** is a section of code that accesses shared data and must be executed by only one process/thread at a time.

Example

$V = V + 1$;

This operation should not be interrupted while it is being executed.

Enter Critical Region

 Access Shared Data

Exit Critical Region

Only one process should be inside the critical region at a time.

Synchronization Mechanisms in Linux

Linux uses:

- **Spinlocks**

- **Mutexes**
- **Semaphores**
- **Atomic Operations**
- **Read-Write Locks**

to protect critical regions and prevent race conditions.

Synchronization is the technique used to coordinate concurrent execution and protect shared resources from race conditions.

Critical Region (Critical Section) is a portion of code that accesses shared data and must not be executed simultaneously by multiple processes or threads.

Race Condition occurs when the correctness of a program depends on the timing or order of execution of multiple processes/threads.

Causes of concurrency

1. **Interrupts:** An interrupt can occur asynchronously at almost any time; interrupting the currently executing code.
2. **Softirqs and tasklets:** Kernel can raise or schedule a softirq or tasklet at almost any time.
3. **Kernel preemption:** Because the kernel is preemptive, one task in the kernel can preempt another.
4. **Sleeping and synchronization with user space:** Task in the kernel can sleep and thus invoke the scheduler, resulting in running of a new process.
5. **Symmetrical multiprocessor:** Two or more processors can execute kernel code at exactly the same time.

Disabling kernel preemption prevents task switching on the current CPU, and disabling interrupts prevents interrupt handlers from running on the current CPU. However, on multiprocessor systems, other CPUs can still access the same shared data, leading to race conditions. Therefore, Linux uses synchronization mechanisms such as spinlocks to protect critical regions across all CPUs.

What is NR_CPUS?

NR_CPUS defines the **maximum number of CPUs/processors that an SMP (Symmetric Multiprocessing) Linux kernel can support.**

```
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$ grep NR_CPUS /boot/config-$(uname -r)
CONFIG_NR_CPUS_RANGE_BEGIN=8192
CONFIG_NR_CPUS_RANGE_END=8192
CONFIG_NR_CPUS_DEFAULT=8192
CONFIG_NR_CPUS=8192
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION$ |
```

Term	Meaning
NR_CPUS	Maximum CPUs supported by the kernel
nr_cpus=	Boot parameter to limit CPUs used
num_online_cpus()	Returns currently online CPUs
Online CPU	CPU available for scheduling tasks

```

sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/NR_CPU$ ls
Makefile      nr_cpus_demo.c    nr_cpus_demo.mod.c
modules.order nr_cpus_demo.ko   nr_cpus_demo.mod.o
Module.symvers nr_cpus_demo.mod  nr_cpus_demo.o
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/NR_CPU$ sudo insmod nr_cpus_
demo.ko
[sudo] password for sethuraj:
Sorry, try again.
[sudo] password for sethuraj:
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/NR_CPU$ lsmod | grep nr_cpus
_demo
nr_cpus_demo      12288  0
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/NR_CPU$ sudo dmesg | tail -5
[437234.464823] Maximum CPUs supported (NR_CPUS) = 8192
[437234.464825] Online CPUs = 3
[437234.464827] Possible CPUs = 3
[437234.464828] Present CPUs = 3
[437234.464830] =====
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/NR_CPU$ |

```

smp_processor_id() in Linux Kernel

smp_processor_id() is a kernel API used to get the **ID of the CPU on which the current code is executin**

it will current processor number on ehich kernel is running.

```
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/CPU_DEMO$ make
make -C /lib/modules/6.8.0-124-generic/build M=/home/sethuraj/SYNCHRONIZATIO
N/CPU_DEMO modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~2
2.04.3) 12.3.0
  You are using:          gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
  CC [M]  /home/sethuraj/SYNCHRONIZATION/CPU_DEMO/cpu_demo.o
  MODPOST /home/sethuraj/SYNCHRONIZATION/CPU_DEMO/Module.symvers
  CC [M]  /home/sethuraj/SYNCHRONIZATION/CPU_DEMO/cpu_demo.mod.o
  LD [M]  /home/sethuraj/SYNCHRONIZATION/CPU_DEMO/cpu_demo.ko
  BTF [M] /home/sethuraj/SYNCHRONIZATION/CPU_DEMO/cpu_demo.ko
Skipping BTF generation for /home/sethuraj/SYNCHRONIZATION/CPU_DEMO/cpu_demo
.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/CPU_DEMO$ nano README.md
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/CPU_DEMO$ sudo insmod cpu_de
mo.ko
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/CPU_DEMO$ sudo dmesg | tail
-5
[437234.464828] Present CPUs = 3
[437234.464830] =====
[437917.518396] Online CPU: 0
[437917.518401] Online CPU: 1
[437917.518402] Online CPU: 2
sethuraj@qbs-generic-lin-2204:~/SYNCHRONIZATION/CPU_DEMO$ |
```